

Chapter 7: API Request Validation

Welcome back to the CinéConnect tutorial! In [Chapter 6: Backend API Core](#), we explored the foundational "brain" of our application, understanding how `Express.js` routes requests, `TypeScript` provides structure, and `Drizzle ORM` seamlessly connects to `PostgreSQL`. Now, before our backend processes any data, we need to ensure that the information it receives is always correct, safe, and exactly what we expect.

🚧 The Strict Bouncer: Why API Request Validation Matters

Imagine our backend as an exclusive club. While our previous chapters built the grand entrance (frontend layout), the VIP area (user authentication), and the amazing dance floor (real-time chat), we still need a very important role: a **strict bouncer** at the door.

This bouncer's job is to inspect everyone trying to get in (every incoming API request) and ensure they meet specific criteria. What if someone tries to register with an email address that isn't really an email, or a password that's too short? What if they try to "like" a movie with an invalid movie ID?

Without a strict bouncer, our backend might try to process this bad information, leading to errors, security vulnerabilities, or corrupted data in our database.

Our main goal in this chapter is to understand how CinéConnect uses a powerful library called `Zod` (our "strict bouncer") to define clear rules for all incoming API requests and automatically check if the data is valid before it's processed.

Let's use a common action as our example: **A user tries to register with invalid details.**

Introducing `Zod`: Our Rulebook for Valid Data

`Zod` is like a meticulously written rulebook that defines exactly what data we expect for every type of request.

- 1. Defining Rules (Schemas):** For every API endpoint (like "register user" or "add comment"), we write a `Zod` "schema." This schema is a set of rules that describe the shape and content of the incoming data (e.g., in the request's body, query parameters, or URL parameters).
 - *Example rule:* "The `email` field must be a valid email format."
 - *Example rule:* "The `password` field must be at least 8 characters long."
 - *Example rule:* "The `movieId` in the URL must be a number."
- 2. Checking Data (Validation):** When a request comes in, our `Zod` validation middleware takes the incoming data and checks it against the defined schema.
 - If all rules are met, the data is considered valid, and the request is allowed to proceed to our backend's logic (controller and service).
 - If any rule is broken, the request is immediately rejected with a clear error message, telling the sender exactly what was wrong.

This process significantly enhances data integrity and security by ensuring only clean and expected data reaches our core backend services.

Use Case: Trying to Register with Invalid Details

Let's walk through an example where **Zod** acts as our bouncer.

The User Experience (Sending Bad Data)

1. A user visits the registration page.
2. They enter "john.doe" as their email (not a valid format).
3. They enter "123" as their password (too short).
4. They click "Register."

Expected Output (Error Response)

Instead of our backend trying to create a user with bad data, the **Zod** bouncer intercepts the request. The user would immediately see an error message like:

```
{
  "success": false,
  "error": "Validation Failed",
  "status": 400,
  "data": {
    "errors": [
      {
        "path": "body.email",
        "message": "Invalid email format"
      },
      {
        "path": "body.password",
        "message": "Password must be at least 8 characters"
      }
    ]
  }
}
```

This response tells the frontend exactly what went wrong with the input, allowing it to display helpful messages to the user. The backend's main logic for user registration is never even touched by this invalid request.

How to Define Rules with **Zod** Schemas (**auth.schema.ts**)

First, let's see how we define these rules using **Zod**. We keep our schemas in dedicated files, like **auth.schema.ts** for authentication-related data.

```
// apps/backend/src/schemas/auth.schema.ts (Simplified)
import { z } from "zod"; // Import Zod

const PASSWORD_MIN_LENGTH = 8;

export const registerSchema = z.object({ // This is our main rulebook for
  registration
    body: z.object({ // Rules for data expected in the request body
```

```

    name: z.string().min(1, "Name is required").max(255), // Name must be a
string, at least 1 char
    email: z.string().email("Invalid email format"), // Email must be a string
and a valid email
    password: z.string().min(PASSWORD_MIN_LENGTH, `Password must be at least
${PASSWORD_MIN_LENGTH} characters`), // Password must be string, min length
  }},
});

```

In this snippet:

- `import { z } from "zod";`: We bring in the `Zod` library.
- `registerSchema = z.object({...});`: This creates a `Zod` schema. It expects the entire incoming request to be an `object`.
- `body: z.object({...});`: Inside this object, we define rules for the `body` of the request. `Zod` automatically looks into `req.body`.
- `name: z.string().min(1, "...").max(255)`: This rule says the `name` field in the request body *must* be a `string`, have a minimum length of 1 character, and a maximum of 255 characters.
- `email: z.string().email("...")`: The `email` field must be a `string` and also conform to a standard email format.
- `password: z.string().min(PASSWORD_MIN_LENGTH, "...")`: The `password` field must be a `string` with a minimum length of 8 characters.

`Zod` gives us many more validators for numbers, dates, booleans, arrays, and more, making it easy to describe complex data structures!

How to Apply Rules with `validate` Middleware (`validation.middleware.ts`)

Once we have our `Zod` schemas (our "rulebooks"), we need a way to apply them to incoming requests. This is where our `validate` middleware comes in. A "middleware" in `Express.js` is like a checkpoint that requests pass through.

```

// apps/backend/src/middlewares/validation.middleware.ts (Simplified)
import { Request, Response, NextFunction } from "express";
import { ZodError, ZodObject } from "zod"; // ZodError helps us identify
validation failures
import { failure } from "../utils/apiResponse"; // Our helper to send error
responses

export const validate = (schema: ZodObject<any, any>) => // Takes a Zod schema as
input
  async (req: Request, res: Response, next: NextFunction) => {
    try {
      await schema.parseAsync({ // Try to validate the request against the
schema
        body: req.body,
        query: req.query,
        params: req.params,

```

```

    });
    return next(); // If valid, let the request proceed to the next
handler
  } catch (error) {
    if (error instanceof ZodError) { // If Zod found issues
      const issues = error.issues; // Get the list of specific
validation errors
      return failure(res, "Validation Failed", 400, {
        errors: issues.map((e) => ({
          path: e.path.join("."), // Where the error occurred (e.g.,
'body.email')
          message: e.message,      // What was wrong (e.g., 'Invalid
email format')
        })),
      });
    }
    // If it's some other unexpected error during validation
    return failure(res, "Internal Server Error during validation", 500);
  }
};

```

This `validate` function takes a `Zod` schema and returns another function, which is the actual `Express` middleware.

- `schema.parseAsync({ body: req.body, ... })`: This is the core `Zod` call. It attempts to validate the `req.body`, `req.query`, and `req.params` against the provided `schema`. If everything matches the rules, it proceeds.
- `try...catch`: If `Zod` finds any issues, `schema.parseAsync` will throw a `ZodError`. Our `catch` block then intercepts this error.
- `failure(res, "Validation Failed", 400, { errors: ... })`: We use our `failure` helper (from [Chapter 6: Backend API Core](#)) to send a standardized `HTTP 400 Bad Request` response, including details about all the validation `errors`.
- `next()`: If validation passes, `next()` is called, allowing the request to proceed to the next middleware or the actual controller function.

How it's Used in a Route (`auth.route.ts`)

Now, let's see how our `registerSchema` and `validate` middleware are put together in a route definition (part of `app.ts` from [Chapter 6: Backend API Core](#)):

```

// apps/backend/src/routes/auth.route.ts (Conceptual)
import { Router } from 'express';
import { AuthController } from '../controllers/auth.controller';
import { validate } from '../middlewares/validation.middleware'; // Our bouncer!
import { registerSchema, loginSchema } from '../schemas/auth.schema'; // Our
rulebooks!

const router = Router();

// This route uses our 'validate' middleware with the 'registerSchema'

```

```
router.post('/register', validate(registerSchema), AuthController.register);
router.post('/login', validate(loginSchema), AuthController.login);
// ... other auth routes ...

export default router;
```

Here, `validate(registerSchema)` is placed *before* `AuthController.register`. This means any incoming request to `/register` will first go through the `validate` middleware. Only if validation succeeds will `AuthController.register` be called to actually process the registration.

Under the Hood: The Validation Flow

Let's trace what happens when an invalid registration request hits our backend: [Validation Flow Diagram](#)

1. **User Initiates Request:** The `User` tries to register with invalid data, and the `Frontend` sends it to the backend.
2. **Express Receives:** Our `Express App` (the traffic controller) receives the `POST /api/v1/auth/register` request.
3. **Validation Checkpoint:** `Express` sees that the `validate(registerSchema)` middleware is the first handler for this route, so it passes the request to the `Validation Middleware`.
4. **Zod Inspects:** Inside the `Validation Middleware`, `Zod` compares the `req.body` data against the `registerSchema` rules.
5. **Rejection:** `Zod` immediately finds that the email is invalid and the password is too short. It throws a `ZodError`.
6. **Error Response:** The `Validation Middleware`'s `catch` block catches the `ZodError`, formats the error details, and sends an `HTTP 400 Bad Request` response back to the `Frontend`.
7. **Frontend Displays Error:** The `Frontend` receives this error and displays user-friendly messages.
8. **Controller Skipped:** Crucially, `AuthController.register` (the function that would actually create the user in the database) is *never* called, protecting our system from bad input.

Other Validation Examples (`movie.schema.ts`)

Let's quickly look at `Zod` schemas for other parts of CinéConnect to demonstrate its flexibility with different types of input:

1. Validating URL Parameters (`params`)

When fetching details for a movie, the `movieId` comes from the URL (e.g., `/movies/550`).

```
// apps/backend/src/schemas/movie.schema.ts (Simplified)
import { z } from "zod";

export const getMovieDetailsSchema = z.object({
  params: z.object({ // Rules for data expected in URL parameters (req.params)
    movieId: z.string().regex(/^\d+$/).transform(Number), // Must be string of
    digits, then convert to number
  }),
});
```

- `params: z.object({...});` Here, `Zod` looks at `req.params`.
- `movieId: z.string().regex(/^\d+$/).transform(Number);` This rule says `movieId` must first be a `string` that consists *only* of digits (`regex(/^\d+$/)`). If it passes that, `transform(Number)` converts it into an actual JavaScript `number` before passing it to the controller. This ensures we only get valid numeric IDs.

2. Validating Request Body Content (`body`)

When a user adds a comment to a movie, the `movieId` and `comment` text are in the request body.

```
// apps/backend/src/schemas/movie.schema.ts (Simplified)
import { z } from "zod";

export const addCommentSchema = z.object({
  body: z.object({ // Rules for data expected in the request body
    movieId: z.number().int().positive(), // Must be a positive integer
    comment: z.string().min(1).max(1000), // Must be a string, between 1 and
1000 characters
  }),
});
```

- `body: z.object({...});` `Zod` checks `req.body` here.
- `movieId: z.number().int().positive();` This ensures the `movieId` is a whole number and greater than zero.
- `comment: z.string().min(1).max(1000);` The comment text must be a string, at least 1 character long, and no more than 1000 characters.

These examples show how `Zod` allows us to define precise expectations for all incoming data, whether it's in the body, URL, or query parameters.

Real-world Validation Tests (`validation.test.ts`)

Our tests confirm that our `Zod` validation is working as expected:

```
// apps/backend/src/tests/validation.test.ts (Simplified)
import { describe, it, expect, beforeEach } from 'vitest';
import request from 'supertest'; // For making HTTP requests in tests
import app from '../app'; // Our Express application

describe('Validation Error Handling', () => {
  // ... (setup for authenticated user) ...

  it('should return 400 when movieId is not numeric in get movie details', async
() => {
    const response = await request(app)
      .get('/api/v1/movies/abc'); // Try to get movie details with 'abc' as
ID
```

```
    expect(response.status).toBe(400); // Expect a Bad Request status
    expect(response.body.success).toBe(false);
    expect(response.body.errors[0].path).toBe('params.movieId'); // Error
    refers to the URL parameter
  });

  it('should return 400 when comment is too short', async () => {
    const response = await request(app)
      .post('/api/v1/movies/comments') // Posting a comment
      .send({ movieId: 550, comment: '' }); // With an empty comment

    expect(response.status).toBe(400);
    expect(response.body.errors[0].path).toBe('body.comment'); // Error refers
    to the comment in the body
  });
});
```

These tests simulate sending invalid requests and assert that our backend correctly responds with a **400 Bad Request** and detailed error messages, proving our **Zod** bouncer is doing its job!

Conclusion

In this chapter, we've explored the critical concept of API request validation in CinéConnect. We learned how **Zod** acts as a strict bouncer, defining clear rules (schemas) for all incoming data, whether it's in the request body, URL parameters, or query strings. By using a **validate** middleware, we ensure that every request is checked against these rules, rejecting invalid or unsafe information early with helpful error messages. This robust validation system is fundamental for maintaining data integrity, enhancing security, and preventing bugs in our backend.

Now that we understand how individual requests are processed and validated, in the next chapter, we'll zoom out to see how all these pieces fit together in our larger project structure with [Monorepo Project Structure](#).